

Firebase Authentication



Most apps need to know the identity of a user. Knowing a user's identity allows an app to securely save user data in the cloud and provide the same personalized experience across all of the user's devices.

Firebase Authentication provides backend services, easy-to-use SDKs, and ready-made UI libraries to authenticate users to your app. It supports authentication using passwords, phone numbers, popular federated identity providers like Google, Facebook and Twitter, and more.

Firebase Authentication integrates tightly with other Firebase services, and it leverages industry standards like OAuth 2.0 and OpenID Connect, so it can be easily integrated with your custom backend.

When you upgrade to Firebase Authentication with Identity Platform (#identity-platform), you unlock additional features, such as multi-factor authentication, blocking functions, user activity and audit logging, SAML and generic OpenID Connect support, multi-tenancy, and enterprise-level support.

Learn how to get started (</docs/auth/where-to-start>)

Key capabilities

You can sign in users to your Firebase app either by using FirebaseUI as a complete drop-in auth solution or by using the Firebase Authentication SDK to manually integrate one or several sign-in methods into your app.

FirebaseUI Auth

Drop-in authentication solution

The recommended way to add a complete sign-in system to your app.

FirebaseUI provides a drop-in auth solution that handles the UI flows for signing in users with email addresses and passwords, phone numbers, and with popular federated identity providers, including Google Sign-In and Facebook Login.

The FirebaseUI Auth component implements best practices for authentication on mobile devices and websites, which can maximize sign-in and sign-up conversion for your app. It also handles edge cases like account recovery and account linking that can be security sensitive and error-prone to handle correctly.

FirebaseUI can be easily customized to fit in with the rest of your app's visual style, and it is open source, so you aren't constrained in realizing the user experience you want.

iOS (</docs/auth/ios/firebaseui>) **Android** (</docs/auth/android/firebaseui>)

Web ([//github.com/firebase/firebaseui-web](https://github.com/firebase/firebaseui-web))

Firebase Authentication SDK

Email and password based authentication

Authenticate users with their email addresses and passwords.

The Firebase Authentication SDK provides methods to create and manage users that use their email addresses and passwords to sign in. Firebase Authentication also handles sending password reset emails.

iOS (</docs/auth/ios/password-auth>)

Android (</docs/auth/android/password-auth>)

Web (</docs/auth/web/password-auth>)

C++ (/docs/auth/cpp/password-auth)

Unity (/docs/auth/unity/password-auth)

Federated identity provider integration

Authenticate users by integrating with federated identity providers. The Firebase Authentication SDK provides methods that allow users to sign in with their Google, Facebook, Twitter, and GitHub accounts.

Sign in with Google	iOS (/docs/auth/ios/google-signin)
	Android (/docs/auth/android/google-signin)
	Web (/docs/auth/web/google-signin)
	C++ (/docs/auth/cpp/google-signin)
	Unity (/docs/auth/unity/google-signin)
Sign in with Apple	iOS (/docs/auth/ios/apple)
	Android (/docs/auth/android/apple)
	Web (/docs/auth/web/apple) C++ (/docs/auth/cpp/apple)
	Unity (/docs/auth/unity/apple)
Facebook	iOS (/docs/auth/ios/facebook-login)
	Android (/docs/auth/android/facebook-login)
	Web (/docs/auth/web/facebook-login)
	C++ (/docs/auth/cpp/facebook-login)
	Unity (/docs/auth/unity/facebook-login)
Twitter	iOS (/docs/auth/ios/twitter-login)
	Android (/docs/auth/android/twitter-login)
	Web (/docs/auth/web/twitter-login)

[C++ \(/docs/auth/cpp/twitter-login\)](/docs/auth/cpp/twitter-login)

[Unity \(/docs/auth/unity/twitter-login\)](/docs/auth/unity/twitter-login)

[iOS \(/docs/auth/ios/github-auth\)](/docs/auth/ios/github-auth)

[Android \(/docs/auth/android/github-auth\)](/docs/auth/android/github-auth)

GitHub [Web \(/docs/auth/web/github-auth\)](/docs/auth/web/github-auth)

[C++ \(/docs/auth/cpp/github-auth\)](/docs/auth/cpp/github-auth)

[Unity \(/docs/auth/unity/github-auth\)](/docs/auth/unity/github-auth)

Phone number authentication

Authenticate users by sending SMS messages to their phones.

[iOS \(/docs/auth/ios/phone-auth\)](/docs/auth/ios/phone-auth)

[Android \(/docs/auth/android/phone-auth\)](/docs/auth/android/phone-auth)

[Web \(/docs/auth/web/phone-auth\)](/docs/auth/web/phone-auth) [C++ \(/docs/auth/cpp/phone-auth\)](/docs/auth/cpp/phone-auth)

[Unity \(/docs/auth/unity/phone-auth\)](/docs/auth/unity/phone-auth)

Custom auth system integration

Connect your app's existing sign-in system to the Firebase Authentication SDK and gain access to Firebase Realtime Database and other Firebase services.

[iOS \(/docs/auth/ios/custom-auth\)](/docs/auth/ios/custom-auth)

[Android \(/docs/auth/android/custom-auth\)](/docs/auth/android/custom-auth)

[Web \(/docs/auth/web/custom-auth\)](/docs/auth/web/custom-auth) [C++ \(/docs/auth/cpp/custom-auth\)](/docs/auth/cpp/custom-auth)

[Unity \(/docs/auth/unity/custom-auth\)](/docs/auth/unity/custom-auth)

Anonymous auth

Use features that require authentication without requiring users to sign in first by creating temporary anonymous accounts. If the user later chooses to sign up, you can upgrade the anonymous account to a regular account, so the user can continue where they left off.

iOS (</docs/auth/ios/anonymous-auth>)

Android (</docs/auth/android/anonymous-auth>)

Web (</docs/auth/web/anonymous-auth>)

C++ (</docs/auth/cpp/anonymous-auth>)

Unity (</docs/auth/unity/anonymous-auth>)

Firebase Authentication with Identity Platform

Firebase Authentication with Identity Platform is an optional upgrade that adds several new features to Firebase Authentication.

This upgrade does not require any migration—your existing client SDK and admin SDK code will continue to work as before, and you'll gain immediate access to features such as enhanced logging and enterprise-grade support and SLAs. With some additional code, you'll be able to add multi-factor auth, blocking functions, and support for SAML and OpenID Connect providers.

Firebase Authentication with Identity Platform has a different pricing scheme compared to the base product. When upgraded, no-cost (Spark) plan projects will be limited to 3,000 daily active users, and pay-as-you-go (Blaze) plan projects will be charged for usage beyond the free tier of 50,000 monthly active users. Be sure you understand the billing implications before you upgrade.

Read more about the new features, pricing, and limits below.

Features

Multi-factor authentication	Multi-factor authentication with SMS protects your users' data by adding a second layer of security to your app. Learn how to add MFA to your Apple (/docs/auth/ios/multi-factor), Android (/docs/auth/android/multi-factor), and web (/docs/auth/web/multi-factor) apps.
Blocking functions	Blocking functions let you run custom code that modifies the result of a user registering or signing in to your app. Learn how to extend Firebase Authentication with blocking functions (/docs/auth/extend-with-blocking-functions).
SAML and OpenID Connect providers	Support sign-in using SAML (web only) and OpenID Connect providers not natively supported by Firebase. Learn how to add SAML sign-in (/docs/auth/web/saml) to web apps and OpenID Connect sign-in to Apple (/docs/auth/ios/openid-connect), Android (/docs/auth/android/openid-connect), and web (/docs/auth/web/openid-connect) apps.
User activity and audit logging	Monitor and log administrative access and end-user activity. When you upgrade your project, you automatically enable admin activity audit logs in Cloud Logging. You can also enable user activity logging on the Authentication Settings (https://console.firebase.google.com/project/_/authentication/settings) page of the Firebase console. To learn how to view and analyze your logs, see the Cloud Logging documentation (https://cloud.google.com/logging/docs).
Multi-tenancy	Using tenants, you can create multiple unique silos of users and configurations within a single project. See Getting started with multi-tenancy (https://cloud.google.com/identity-platform/docs/multi-tenancy-quickstart) in the Cloud Identity Platform documentation.
Enterprise support and SLA	Upgraded projects get uptime commitments for Auth services according to the Identity Platform Service Level Agreement (SLA) (https://cloud.google.com/identity-platform/sla) and are eligible to upgrade to enterprise-grade support.

Automatic clean-up of anonymous users	You will get the option to enable anonymous accounts to be automatically deleted if they are over thirty days old. Anonymous accounts also will no longer count towards billing and usage quotas.
---------------------------------------	---

Usage limits

Upon upgrade, Firebase Authentication with Identity Platform introduces new limits to your use of Firebase Authentication.

No cost (Spark)

Projects on the no-cost (Spark) plan have a new limit of 3,000 daily active users (DAUs) for most sign-in providers. Daily active usage is calculated based on how many unique users sign in during a 24 hour period.

Providers	New limit	Old limit
Email, Social, Anonymous, Custom	3,000 DAUs	Unlimited
SAML, OpenID Connect	2 DAUs	N/A

Pay as you go (Blaze)

Pricing for projects on the Blaze plan is based on monthly active users (MAUs) and includes a no-cost tier of 50,000 users. An active user is anyone who uses their account within the billing period.

Providers	No-cost tier	Cost (\$) per MAU above no-cost tier
Email, Social, Anonymous, Custom	0-49,999 MAUs	0.0025 to 0.0055 per MAU
SAML, OpenID Connect	0-49 MAUs	0.015 per MAU

Upgrade your project

To upgrade your project to Firebase Authentication with Identity Platform, open the Authentication Settings (https://console.firebase.google.com/project/_/authentication/settings) page of

the Firebase console.

How does it work?

To sign a user into your app, you first get authentication credentials from the user. These credentials can be the user's email address and password, or an OAuth token from a federated identity provider. Then, you pass these credentials to the Firebase Authentication SDK. Our backend services will then verify those credentials and return a response to the client.



After a successful sign in, you can access the user's basic profile information, and you can control the user's access to data stored in other Firebase products. You can also use the provided authentication token to verify the identity of users in your own backend services.

Note: By default, authenticated users can read and write data to the Firebase Realtime Database and Cloud Storage. You can control the access of those users by modifying your [Firebase Realtime Database](/docs/database/security) (/docs/database/security) and [Cloud Storage Security Rules](/docs/storage/security) (/docs/storage/security).

Implementation paths

Using FirebaseUI Auth

1 Set up sign-in methods

For email address and password or phone number sign-in and any federated identity providers you want to support, enable them in the Firebase console and complete any configuration required by the identity provider, such as setting your OAuth redirect URL.

2 Customize the sign-in UI

You can customize the sign-in UI by setting FirebaseUI options, or fork the code on GitHub to customize the sign-in experience further.

3 Use FirebaseUI to perform the sign-in flow

Import the FirebaseUI library, specify the sign-in methods you want to support, and initiate the FirebaseUI sign-in flow.

Using the Firebase Authentication SDK

1 Set up sign-in methods

For email address and password or phone number sign-in and any federated identity providers you want to support, enable them in the Firebase console and complete any configuration required by the identity provider, such as setting your OAuth redirect URL.

2 Implement UI flows for your sign-in methods

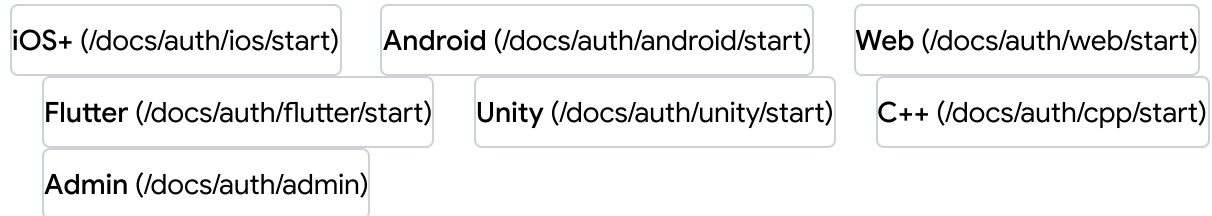
For email address and password sign-in, implement a flow that prompts users to type their email addresses and passwords. For phone number sign-in, create a flow that prompts users for their phone number, and then for the code from the SMS message they receive. For federated sign-in, implement the flow required by each provider.

3 Pass the user's credentials to the Firebase Authentication SDK

Pass the user's email address and password or the OAuth token that was acquired from the federated identity provider to the Firebase Authentication SDK.

What's next

Learn more about [users in a Firebase project](/docs/auth/users) (/docs/auth/users), then check out the getting started guides for the platform and sign-in providers you want to support:



Not sure where to begin?

[Learn how to get started](/docs/auth/where-to-start) (/docs/auth/where-to-start)

Except as otherwise noted, the content of this page is licensed under the [Creative Commons Attribution 4.0 License](https://creativecommons.org/licenses/by/4.0/) (https://creativecommons.org/licenses/by/4.0/), and code samples are licensed under the [Apache 2.0 License](https://www.apache.org/licenses/LICENSE-2.0) (https://www.apache.org/licenses/LICENSE-2.0). For details, see the [Google Developers Site Policies](https://developers.google.com/site-policies) (https://developers.google.com/site-policies). Java is a registered trademark of Oracle and/or its affiliates.

Last updated 2026-05-19 UTC.